

four ch 1 exercises

2025-08-03

1.1

```
x <- c(1, 2, 3)
y <- c(6, 5, 4)
```

Predict what will happen when the pieces of code are run.

- a. `x * 2` Each element in `x` is doubled 2 4 6

```
x * 2
```

```
## [1] 2 4 6
```

- b. `x * y` Each element in `x` is multiplied by the corresponding element in `y` 6 10 12

```
x * y
```

```
## [1] 6 10 12
```

- c. `x[1] * y[2]` First element of `x` is multiplied by second element of `y` 5

```
x[1] * y[2]
```

```
## [1] 5
```

1.2 What is the value of `x` after each of the following commands? a. `x + x` `x` is still 1 2 3 because the new value of `x + x` is not reassigned to `x`

- b. `x <- x + x` `x` becomes 2 4 6 as each element in `x` gets incremented by itself (effectively doubled)

- c. `y <- x + x` `x` is still 1 2 3 because `x + x` is assigned to `y` not `x`

- d. `x <- x + 1` `x` becomes 2 3 4 as each element gets incremented by 1

1.3 Determine the values of the vector `vec` after each of the following commands is run.

- a. `vec <- 1:10` `vec` becomes 1 2 3 4 5 6 7 8 9 10

- b. `vec <- 1:10 * 2` `vec` becomes 2 4 6 8 10 12 14 16 18 20

- c. `vec <- 1:10^2` `vec` becomes 1 2 3 4 ... 99 100

- d. `vec <- 1:10 + 1` `vec` becomes 2 3 4 5 6 7 8 9 10 11
- e. `vec <- 1:(10 * 2)` `vec` becomes 1 2 3 4 ... 19 20
- f. `vec <- rep(c(1,1,2), times = 2)` `vec` becomes 1 1 2 1 1 2
- g. `vec <- seq(from = 0, to = 10, length.out = 5)` `vec` becomes 0.0 2.5 5.0 7.5 10.0

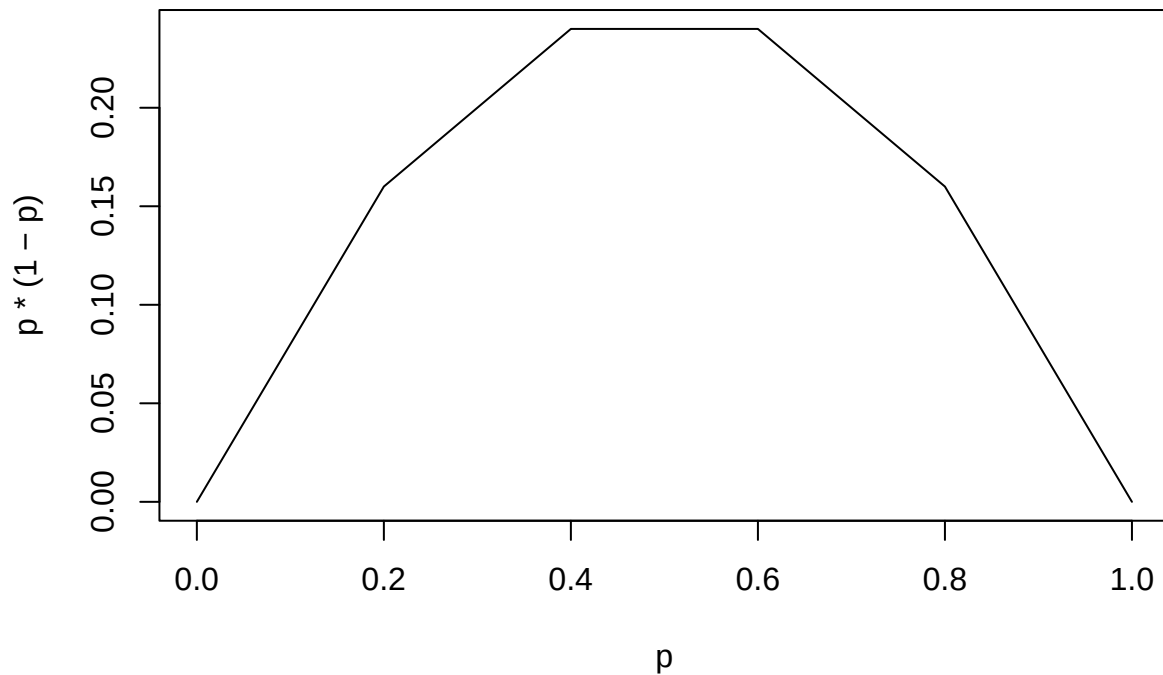
1.4 Graph the function $f(p) = p(1 - p)$ for $p \in [0, 1]$.

- a. Use `seq` to create a vector `p` of numbers from 0 to 1 spaced by 0.2

```
p <- seq(from = 0, to = 1, by = 0.2)
```

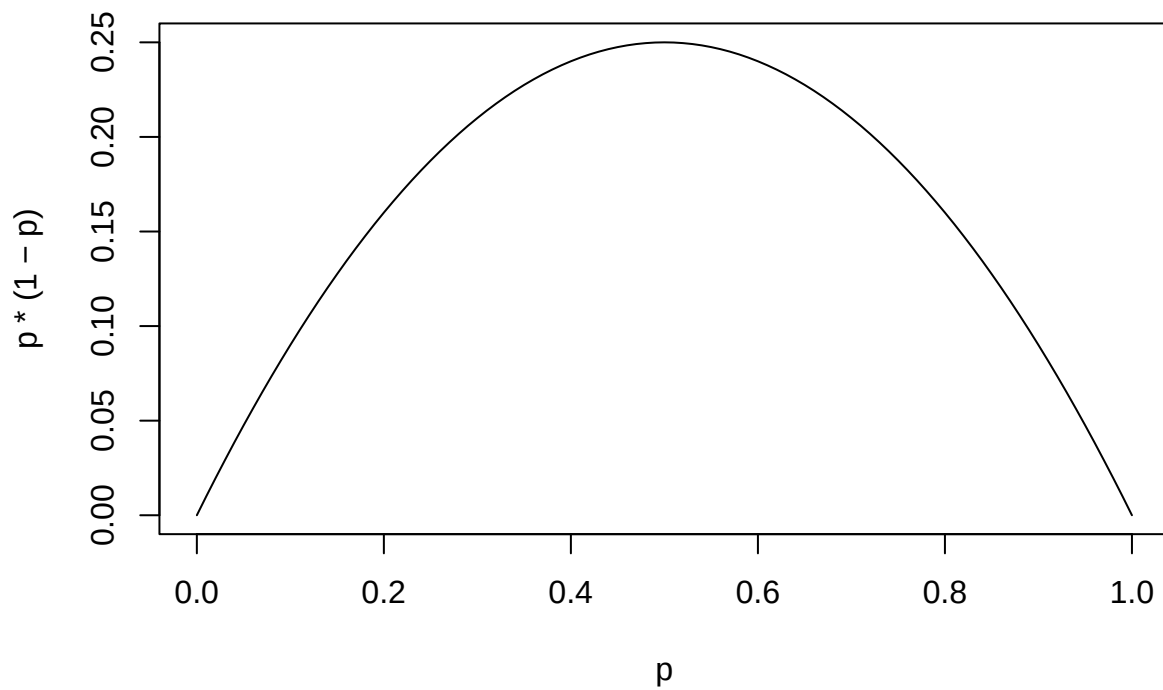
- b. Use `plot` to plot `p` in the x coordinate and `p(1 - p)` in the y coordinate.

```
plot(x = p, y = p * (1 - p), type = "l")
```



- c. Repeat, but with a vector `p` of numbers from 0 to 1 spaced by 0.01

```
p <- seq(0, 1, by = 0.01)
plot(x = p, y = p * (1 - p), type = "l")
```



1.5 Use R to calculate the sum of the squares of all numbers from 1 to 100: $1^2 + 2^2 + \dots + 99^2 + 100^2$.

```
sum((1:100)^2)
```

```
## [1] 338350
```

1.6

```
x <- seq(from = 10, to = 30, by = 2)
```

a. What is the length of `x` (number of elements)?

```
str(x)
```

```
##  num [1:11] 10 12 14 16 18 20 22 24 26 28 ...
```

```
length(x)
```

```
## [1] 11
```

b. What is `x[2]`? 12

c. What is `x[1:5]`? 10 12 14 16 18

- d. What is `x[1:3*2]`? Equivalent to `x[c(2, 4, 6)]` 12 16 20
- e. What is `x[1:(3*2)]`? Equivalent to `x[1:6]` 10 12 14 16 18 20
- f. What is `x > 25`? FALSE x8 TRUE x3

```
x > 25
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE TRUE TRUE
```

- g. What is `x[x > 25]`? 26 28 30
- h. What is `x[-1]`? 12 14 16 18 20 22 24 26 28 30
- i. What is `x[-1:-3]` Equivalent to `x[c(-1, -2, -3)]` 16 18 20 22 24 26 28 30

1.7 R has built in vector `rivers` which contains the length of major North American rivers.

- a. Use `?rivers` to learn about the dataset.

```
?rivers
```

Contains 141 observations

- b. Find the mean and standard deviation of the rivers data.

```
# is.na(rivers)
mean(rivers)
```

```
## [1] 591.1844
```

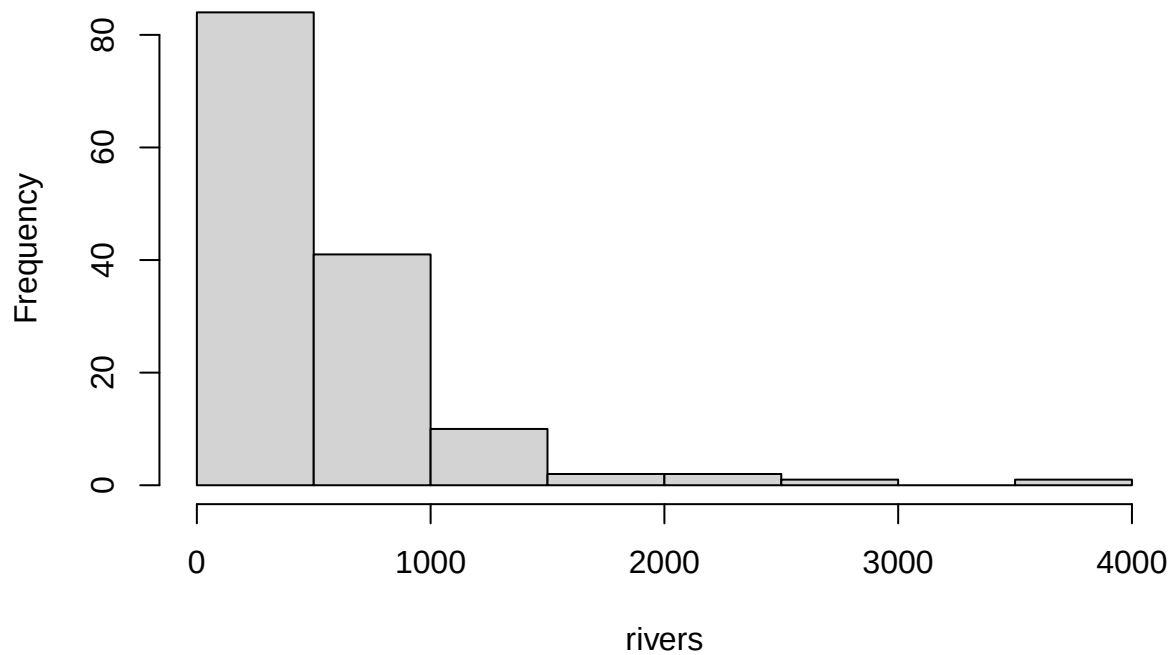
```
sd(rivers)
```

```
## [1] 493.8708
```

- c. Make a histogram (`hist`) of the rivers data.

```
hist(rivers)
```

Histogram of rivers



d. Get the five number summary (`summary`) of the rivers data.

```
summary(rivers)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  135.0   310.0   425.0   591.2   680.0  3710.0
```

e. Find the longest and shortest length of rivers in the set.

```
max(rivers)
```

```
## [1] 3710
```

```
min(rivers)
```

```
## [1] 135
```

f. Make a list of all (lengths of) rivers longer than 1000 miles.

```
rivers[rivers > 1000]
```

```
## [1] 1459 1450 1243 2348 1171 3710 2315 2533 1306 1054 1270 1885 1100 1205 1038
## [16] 1770
```

1.8 Consider the built-in data frame `airquality`.

- a. How many observations of how many variables are there?

```
str(airquality)
```

```
## 'data.frame': 153 obs. of 6 variables:
## $ Ozone : int 41 36 12 18 NA 28 23 19 8 NA ...
## $ Solar.R: int 190 118 149 313 NA NA 299 99 19 194 ...
## $ Wind : num 7.4 8 12.6 11.5 14.3 14.9 8.6 13.8 20.1 8.6 ...
## $ Temp : int 67 72 74 62 56 66 65 59 61 69 ...
## $ Month : int 5 5 5 5 5 5 5 5 5 5 ...
## $ Day : int 1 2 3 4 5 6 7 8 9 10 ...
```

153 observations of 6 variables

- b. What are the names of the variables? Ozone, Solar.R, Wind, Temp, Month, Day
- c. What type of data is each variable? All are integers except Wind which is numeric
- d. Do you agree with the data type given to each variable? What would have been some alternative choices? Besides Month and Day, the rest could be numeric to capture more precise information

1.9 There is a built-in data set `state`, which is really seven separate variables with names such as `state.name`, `state.region`, and `state.area`.

- a. What are the possible regions a state can be in? How many states are in each region?

```
str(state.region)
```

```
## Factor w/ 4 levels "Northeast","South",...: 2 4 4 2 4 4 1 2 2 2 ...
```

```
levels(state.region)
```

```
## [1] "Northeast" "South" "North Central" "West"
```

```
table(state.region)
```

```
## state.region
## Northeast South North Central West
##          9      16          12      13
```

- b. Which states have area less than 10,000 square miles?

```
state.name[state.area < 10000]
```

```
## [1] "Connecticut" "Delaware" "Hawaii" "Massachusetts"
## [5] "New Hampshire" "New Jersey" "Rhode Island" "Vermont"
```

- c. Which state's geographic center is furthest south? (use `which.min`)

```
str(state.center)
```

```
## List of 2  
## $ x: num [1:50] -86.8 -127.2 -111.6 -92.3 -119.8 ...  
## $ y: num [1:50] 32.6 49.2 34.2 34.7 36.5 ...
```

```
state.name[which.min(state.center$y)]
```

```
## [1] "Florida"
```

1.10 Consider the `mtcars` data set.

a. Which cars have 4 forward gears?

```
str(mtcars)
```

```
## 'data.frame': 32 obs. of 11 variables:  
## $ mpg : num 21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...  
## $ cyl : num 6 6 4 6 8 6 8 4 4 6 ...  
## $ disp: num 160 160 108 258 360 ...  
## $ hp : num 110 110 93 110 175 105 245 62 95 123 ...  
## $ drat: num 3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...  
## $ wt : num 2.62 2.88 2.32 3.21 3.44 ...  
## $ qsec: num 16.5 17 18.6 19.4 17 ...  
## $ vs : num 0 0 1 1 0 1 0 1 1 1 ...  
## $ am : num 1 1 1 0 0 0 0 0 0 0 ...  
## $ gear: num 4 4 4 3 3 3 3 4 4 4 ...  
## $ carb: num 4 4 1 1 1 2 1 4 2 2 4 ...
```

```
mtcars[mtcars$gear == 4,]
```

```
##      mpg  cyl  disp  hp drat   wt  qsec vs am gear carb  
## Mazda RX4      21.0   6 160.0 110 3.90 2.620 16.46 0 1    4    4  
## Mazda RX4 Wag  21.0   6 160.0 110 3.90 2.875 17.02 0 1    4    4  
## Datsun 710     22.8   4 108.0  93 3.85 2.320 18.61 1 1    4    1  
## Merc 240D      24.4   4 146.7  62 3.69 3.190 20.00 1 0    4    2  
## Merc 230       22.8   4 140.8  95 3.92 3.150 22.90 1 0    4    2  
## Merc 280       19.2   6 167.6 123 3.92 3.440 18.30 1 0    4    4  
## Merc 280C      17.8   6 167.6 123 3.92 3.440 18.90 1 0    4    4  
## Fiat 128       32.4   4  78.7  66 4.08 2.200 19.47 1 1    4    1  
## Honda Civic    30.4   4  75.7  52 4.93 1.615 18.52 1 1    4    2  
## Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90 1 1    4    1  
## Fiat X1-9      27.3   4  79.0  66 4.08 1.935 18.90 1 1    4    1  
## Volvo 142E     21.4   4 121.0 109 4.11 2.780 18.60 1 1    4    2
```

b. What subset of `mtcars` does `mtcars[mtcars$disp > 150 & mtcars$mpg > 20,]` describe? Cars with over 150 cubic inches of displacement and greater than 20 miles per gallon

c. Which cars have 4 forward gears and manual transmission? Note: manual transmission is 1 and automatic is 0.

```
mtcars[mtcars$gear == 4 & mtcars$am == 1,]
```

```
##           mpg cyl  disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4      21.0   6 160.0 110 3.90 2.620 16.46 0  1    4    4
## Mazda RX4 Wag  21.0   6 160.0 110 3.90 2.875 17.02 0  1    4    4
## Datsun 710      22.8   4 108.0  93 3.85 2.320 18.61 1  1    4    1
## Fiat 128        32.4   4  78.7  66 4.08 2.200 19.47 1  1    4    1
## Honda Civic     30.4   4  75.7  52 4.93 1.615 18.52 1  1    4    2
## Toyota Corolla  33.9   4  71.1  65 4.22 1.835 19.90 1  1    4    1
## Fiat X1-9       27.3   4  79.0  66 4.08 1.935 18.90 1  1    4    1
## Volvo 142E      21.4   4 121.0 109 4.11 2.780 18.60 1  1    4    2
```

d. Which cars have 4 forward gears or manual transmission?

```
mtcars[mtcars$gear == 4 | mtcars$am == 1,]
```

```
##           mpg cyl  disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4      21.0   6 160.0 110 3.90 2.620 16.46 0  1    4    4
## Mazda RX4 Wag  21.0   6 160.0 110 3.90 2.875 17.02 0  1    4    4
## Datsun 710      22.8   4 108.0  93 3.85 2.320 18.61 1  1    4    1
## Merc 240D       24.4   4 146.7  62 3.69 3.190 20.00 1  0    4    2
## Merc 230        22.8   4 140.8  95 3.92 3.150 22.90 1  0    4    2
## Merc 280        19.2   6 167.6 123 3.92 3.440 18.30 1  0    4    4
## Merc 280C       17.8   6 167.6 123 3.92 3.440 18.90 1  0    4    4
## Fiat 128        32.4   4  78.7  66 4.08 2.200 19.47 1  1    4    1
## Honda Civic     30.4   4  75.7  52 4.93 1.615 18.52 1  1    4    2
## Toyota Corolla  33.9   4  71.1  65 4.22 1.835 19.90 1  1    4    1
## Fiat X1-9       27.3   4  79.0  66 4.08 1.935 18.90 1  1    4    1
## Porsche 914-2   26.0   4 120.3  91 4.43 2.140 16.70 0  1    5    2
## Lotus Europa    30.4   4  95.1 113 3.77 1.513 16.90 1  1    5    2
## Ford Pantera L  15.8   8 351.0 264 4.22 3.170 14.50 0  1    5    4
## Ferrari Dino    19.7   6 145.0 175 3.62 2.770 15.50 0  1    5    6
## Maserati Bora    15.0   8 301.0 335 3.54 3.570 14.60 0  1    5    8
## Volvo 142E      21.4   4 121.0 109 4.11 2.780 18.60 1  1    4    2
```

e. Find the mean mpg of the cars with two carburetors.

```
mean(mtcars[mtcars$carb == 2, 1])
```

```
## [1] 22.4
```

1.11 Consider the `mtcars` data set.

a. Convert the `am` variable to a factor with two levels, `auto` and `manual`.

```
mtcars$am <- factor(mtcars$am, levels = c(0, 1), labels = c("auto", "manual"))
str(mtcars$am)
```

```
## Factor w/ 2 levels "auto","manual": 2 2 2 1 1 1 1 1 1 1 ...
```

b. How many cars of each type of transmission are there?

```
table(mtcars$am)
```

```
##  
##   auto manual  
##    19     13
```

c. How many cars of each type of transmission have gas mileage estimates greater than 25 mpg?

```
table(mtcars[mtcars$mpg > 25, "am"])
```

```
##  
##   auto manual  
##     0      6
```

1.12 This problem uses the data set `hot_dogs` from the package `fosdata`.

a. How many kinds of observations are there? What types are the variables?

```
str(fosdata::hot_dogs)
```

```
## 'data.frame':   54 obs. of  3 variables:  
## $ type      : Factor w/ 3 levels "Beef","Meat",...: 1 1 1 1 1 1 1 1 1 1 ...  
## $ calories: int  186 181 176 149 184 190 158 139 175 148 ...  
## $ sodium   : int  495 477 425 322 482 587 370 322 479 375 ...
```

b. What are the three kinds of hot dogs in this data set?

```
levels(fosdata::hot_dogs$type)
```

```
## [1] "Beef"    "Meat"    "Poultry"
```

c. What is the highest sodium content of any hot dog in this data set?

```
max(fosdata::hot_dogs$sodium)
```

```
## [1] 645
```

d. What is the mean calorie content for Beef hot dogs?

```
mean(fosdata::hot_dogs[fosdata::hot_dogs$type == "Beef", "calories"])
```

```
## [1] 156.85
```

1.13 This problem uses the data set `DrinksWages` from the package `HistData`.

```
library(HistData)
```

- a. How many observations of how many variables are there? What types are the variables?

```
str(DrinksWages)
```

```
## 'data.frame': 70 obs. of 6 variables:
## $ class : Factor w/ 3 levels "A","B","C": 1 1 1 1 1 1 1 1 1 ...
## $ trade : Factor w/ 70 levels "baker","barman",...: 38 10 25 55 36 44 68 34 14 11 ...
## $ sober : int 1 1 2 1 2 9 8 3 0 12 ...
## $ drinks: int 1 10 1 5 0 8 2 5 7 23 ...
## $ wage : num 24 18.4 21.5 21.2 19 ...
## $ n : int 2 11 3 6 2 17 10 8 7 35 ...
```

- b. The variable `wage` contains the average wage for each profession. Which profession has the lowest wage?

```
DrinksWages[which.min(DrinksWages$wage), "trade"]
```

```
## [1] factory worker
## 70 Levels: baker barman billposter blacksmith bookbinder ... wireworker
```

- c. The variable `n` contains the number of workers surveyed for each profession. Sum this to find the total number of workers surveyed.

```
sum(DrinksWages$n)
```

```
## [1] 604
```

- d. Compute the mean wage for all workers surveyed by multiplying `wage * n` for each profession, summing, and dividing by the total number of workers surveyed.

```
sum(DrinksWages$wage * DrinksWages$n) / sum(DrinksWages$n)
```

```
## [1] 24.59782
```

1.14 This problem uses the data set `Batting` from the `Lahman` package which contains batting statistics of all MLB players since 1871, broken down by season.

```
# install.packages("Lahman")
library(Lahman)
```

- a. How many observations of how many variables are there?

```
dim(Batting)
```

```
## [1] 113799 22
```

```
str(Batting)
```

```
## 'data.frame': 113799 obs. of 22 variables:
## $ playerID: chr "aardsda01" "aardsda01" "aardsda01" "aardsda01" ...
## $ yearID : int 2004 2006 2007 2008 2009 2010 2012 2013 2015 1954 ...
## $ stint : int 1 1 1 1 1 1 1 1 1 1 ...
## $ teamID : Factor w/ 149 levels "ALT","ANA","ARI",...: 117 35 33 16 116 116 93 94 4 80 ...
## $ lgID : Factor w/ 7 levels "AA","AL","FL",...: 5 5 2 2 2 2 5 5 5 ...
## $ G : int 11 45 25 47 73 53 1 43 33 122 ...
## $ AB : int 0 2 0 1 0 0 0 0 1 468 ...
## $ R : int 0 0 0 0 0 0 0 0 0 58 ...
## $ H : int 0 0 0 0 0 0 0 0 0 131 ...
## $ X2B : int 0 0 0 0 0 0 0 0 0 27 ...
## $ X3B : int 0 0 0 0 0 0 0 0 0 6 ...
## $ HR : int 0 0 0 0 0 0 0 0 0 13 ...
## $ RBI : int 0 0 0 0 0 0 0 0 0 69 ...
## $ SB : int 0 0 0 0 0 0 0 0 0 2 ...
## $ CS : int 0 0 0 0 0 0 0 0 0 2 ...
## $ BB : int 0 0 0 0 0 0 0 0 0 28 ...
## $ SO : int 0 0 0 1 0 0 0 0 1 39 ...
## $ IBB : int 0 0 0 0 0 0 0 0 0 NA ...
## $ HBP : int 0 0 0 0 0 0 0 0 0 3 ...
## $ SH : int 0 1 0 0 0 0 0 0 0 6 ...
## $ SF : int 0 0 0 0 0 0 0 0 0 4 ...
## $ GIDP : int 0 0 0 0 0 0 0 0 0 13 ...
```

b. Use the command `head(Batting)` to get a look at the first six lines of data.

```
head(Batting)
```

```
## playerID yearID stint teamID lgID G AB R H X2B X3B HR RBI SB CS BB SO IBB
## 1 aardsda01 2004 1 SFN NL 11 0 0 0 0 0 0 0 0 0 0 0 0
## 2 aardsda01 2006 1 CHN NL 45 2 0 0 0 0 0 0 0 0 0 0 0
## 3 aardsda01 2007 1 CHA AL 25 0 0 0 0 0 0 0 0 0 0 0 0
## 4 aardsda01 2008 1 BOS AL 47 1 0 0 0 0 0 0 0 0 0 0 1 0
## 5 aardsda01 2009 1 SEA AL 73 0 0 0 0 0 0 0 0 0 0 0 0 0
## 6 aardsda01 2010 1 SEA AL 53 0 0 0 0 0 0 0 0 0 0 0 0 0
## HBP SH SF GIDP
## 1 0 0 0 0
## 2 0 1 0 0
## 3 0 0 0 0
## 4 0 0 0 0
## 5 0 0 0 0
## 6 0 0 0 0
```

c. What is the most number of triples (X3B) that have been hit in a single season?

```
max(Batting$X3B)
```

```
## [1] 36
```

- d. What is the playerID(s) of the person(s) who hit the most number of triples in a single season? In what year did it happen?

```
Batting[which.max(Batting$X3B), c("playerID", "yearID")]
```

```
##      playerID yearID
## 110751 wilsoch01  1912
```

- e. Which player hit the most number of triples in a single season since 1960?

```
People[People$playerID == Batting[Batting$yearID > 1960,][which.max(Batting[Batting$yearID > 1960, "X3B")],
```

```
##      nameFirst  nameLast
## 7085      Curtis Granderson
```

1.15 Consider the `bechdel` data set in the `fosdata` package.

```
str(fosdata::bechdel)
```

```
## 'data.frame':   1794 obs. of  15 variables:
## $ year      : int   2013 2012 2013 2013 2013 2013 2013 2013 2013 2013 ...
## $ imdb      : chr   "tt1711425" "tt1343727" "tt2024544" "tt1272878" ...
## $ title     : chr   "21 & Over" "Dredd 3D" "12 Years a Slave" "2 Guns" ...
## $ test      : Factor w/ 10 levels "dubious","dubious-disagree",...: 5 10 6 5 3 3 5 10 9 5 ...
## $ clean_test : Factor w/ 5 levels "dubious","men",...: 3 5 3 3 2 2 3 5 5 3 ...
## $ binary    : Factor w/ 2 levels "FAIL","PASS": 1 2 1 1 1 1 1 2 2 1 ...
## $ budget    : num   1.30e+07 4.50e+07 2.00e+07 6.10e+07 4.00e+07 2.25e+08 9.20e+07 1.20e+07 1.30e+07 ...
## $ domgross  : num   25682380 13414714 53107035 75612460 95020213 ...
## $ intgross  : num   4.22e+07 4.09e+07 1.59e+08 1.32e+08 9.50e+07 ...
## $ code      : Factor w/ 85 levels "1970PASS","1971FAIL",...: 84 83 84 84 84 84 84 85 85 84 ...
## $ budget_2013 : num   13000000 45658735 20000000 61000000 40000000 ...
## $ domgross_2013: num   25682380 13611086 53107035 75612460 95020213 ...
## $ intgross_2013: num   4.22e+07 4.15e+07 1.59e+08 1.32e+08 9.50e+07 ...
## $ period_code : int    1 1 1 1 1 1 1 1 1 ...
## $ decade_code : int    1 1 1 1 1 1 1 1 1 ...
```

- a. How many movies in the data set pass the Bechdel test?

```
# nrow(fosdata::bechdel[fosdata::bechdel$binary == "PASS",])
sum(fosdata::bechdel$binary == "PASS")
```

```
## [1] 803
```

- b. What percentage of movies in the data set pass the Bechdel test?

```
mean(fosdata::bechdel$binary == "PASS")
```

```
## [1] 0.4476031
```

- c. Create a table of number of movies in the data set by year.

```
table(fosdata::bechdel$year)
```

```
##
## 1970 1971 1972 1973 1974 1975 1976 1977 1978 1979 1980 1981 1982 1983 1984 1985
##    1    5    3    5    7    5    8    7    8    5   14    9   14    5   16   10
## 1986 1987 1988 1989 1990 1991 1992 1993 1994 1995 1996 1997 1998 1999 2000 2001
##   10   14   19   14   15   13   20   16   26   36   42   51   62   56   63   64
## 2002 2003 2004 2005 2006 2007 2008 2009 2010 2011 2012 2013
##   80   64   81  100   90   73  101  124  129  124   86   99
```

d. Which year has the most movies in the data set?

```
which.max(table(fosdata::bechdel$year))
```

```
## 2010
##    41
```

e. How many different values are there in the `clean_test` variable?

```
length(unique(fosdata::bechdel$clean_test))
```

```
## [1] 5
```

f. Create a data frame that contains only those observations that pass the Bechdel test.

```
pass <- fosdata::bechdel[fosdata::bechdel$binary == "PASS",]
rownames(pass) <- 1:nrow(pass) # reset index
head(pass)
```

```
##   year      imdb          title      test clean_test binary  budget
## 1 2012 tt1343727      Dredd 3D ok-disagree      ok    PASS 4.5e+07
## 2 2013 tt2194499      About Time ok-disagree      ok    PASS 1.2e+07
## 3 2013 tt1814621      Admission      ok      ok    PASS 1.3e+07
## 4 2013 tt1800241      American Hustle ok-disagree      ok    PASS 4.0e+07
## 5 2013 tt1322269 August: Osage County      ok      ok    PASS 2.5e+07
## 6 2013 tt1559547      Beautiful Creatures      ok      ok    PASS 5.0e+07
##   domgross  intgross      code budget_2013 domgross_2013  intgross_2013
## 1  13414714  40868994 2012PASS   45658735    13611086     41467257
## 2  15323921  87324746 2013PASS   12000000     15323921     87324746
## 3  18007317  18007317 2013PASS   13000000     18007317     18007317
## 4 148430908 249484909 2013PASS   40000000    148430908    249484909
## 5   37304874  50304874 2013PASS   25000000     37304874     50304874
## 6  19452138  55940671 2013PASS   50000000     19452138     55940671
##   period_code decade_code
## 1           1           1
## 2           1           1
## 3           1           1
## 4           1           1
## 5           1           1
## 6           1           1
```

- g. Create a data frame that contains all of the observations that do not have missing values in the domgross variable.

```
head(fosdata::bechdel[!is.na(fosdata::bechdel$domgross),])
```

```
##   year      imdb      title      test clean_test binary  budget
## 1 2013 tt1711425  21 & Over      notalk      notalk  FAIL 1.30e+07
## 2 2012 tt1343727      Dredd 3D    ok-disagree      ok    PASS 4.50e+07
## 3 2013 tt2024544 12 Years a Slave notalk-disagree      notalk  FAIL 2.00e+07
## 4 2013 tt1272878      2 Guns      notalk      notalk  FAIL 6.10e+07
## 5 2013 tt0453562      42          men          men  FAIL 4.00e+07
## 6 2013 tt1335975    47 Ronin      men          men  FAIL 2.25e+08
##   domgross  intgross      code budget_2013 domgross_2013 intgross_2013
## 1 25682380 42195766 2013FAIL    13000000    25682380    42195766
## 2 13414714 40868994 2012PASS    45658735    13611086    41467257
## 3 53107035 158607035 2013FAIL    20000000    53107035    158607035
## 4 75612460 132493015 2013FAIL    61000000    75612460    132493015
## 5 95020213 95020213 2013FAIL    40000000    95020213    95020213
## 6 38362475 145803842 2013FAIL    225000000    38362475    145803842
##   period_code decade_code
## 1           1           1
## 2           1           1
## 3           1           1
## 4           1           1
## 5           1           1
## 6           1           1
```